

前情概要

GShard是针对模型训练的开销来考量，提出了硬约束的专家容量和软约束的辅助损失函数，在此基础上，利用随机路由来对专家二调整，达到门控机制的合理性

GShard还将MoE和transformer融合

挑战

Switch旨在升级MoE，调整该算法的复杂性和训练不稳定性

MoE优点在于可以不断增大模型，计算量不会随之线性增长。稀疏性导致的。

稀疏性的想法源自于抛弃了传统**特征复用**，在神经网络内部进行了一个分工，有点像人脑的思维区域分工。

模型改进

专家top-one -> switch层

- 传统观点

1. 如果只选择一个专家，那么top1选择了一个FFN来进行决策。然而，直接挑选该专家断掉了梯度的传播！门控机制输出的是一个专家的权重，如果直接挑选该专家，那么梯度将无法传递到专家之前。如果对权重归一化，那可以认为没有任何意义。
2. 综合考量多个专家，使用更多的参数，直觉认为信息更多，模型更好。

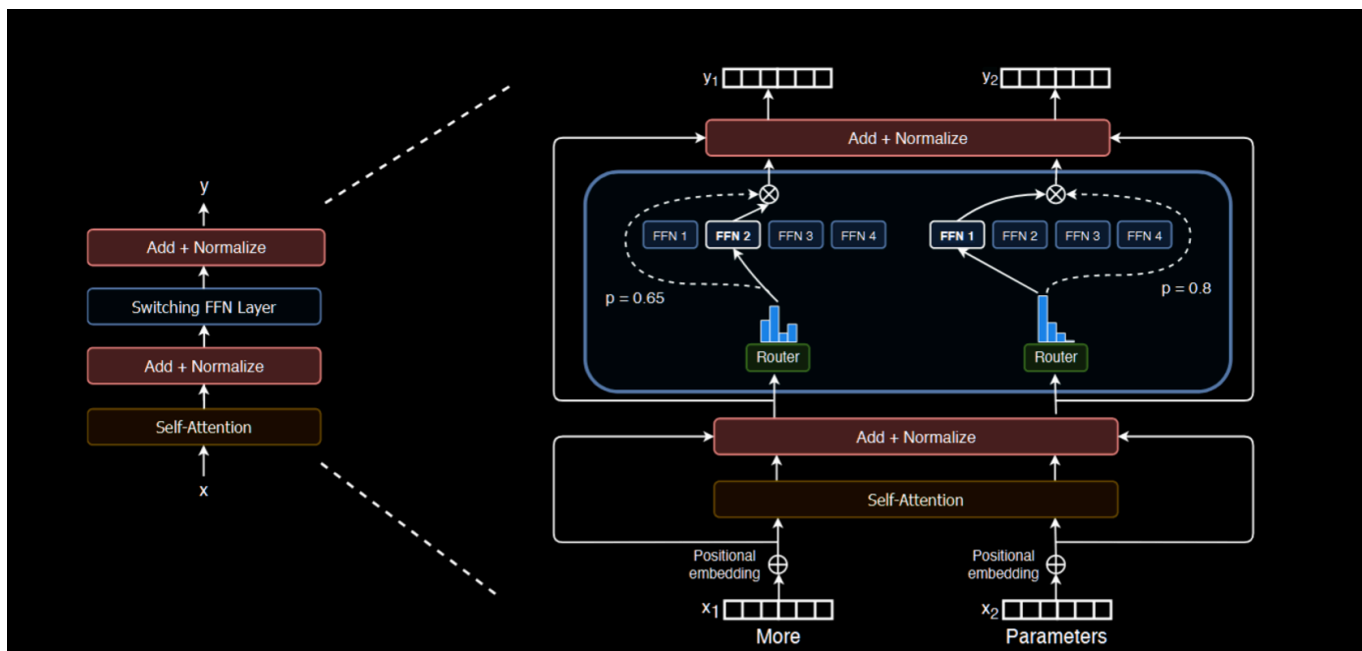
- 改进

只选择一个专家，该专家直接乘上权重，add之后作为结果出来

- 优点

1. 只选择一个专家能够减少多卡之间的沟通成本
2. 能够减少计算量
3. 专家容量能够减少，减少内存占用

模型图片如下



硬约束

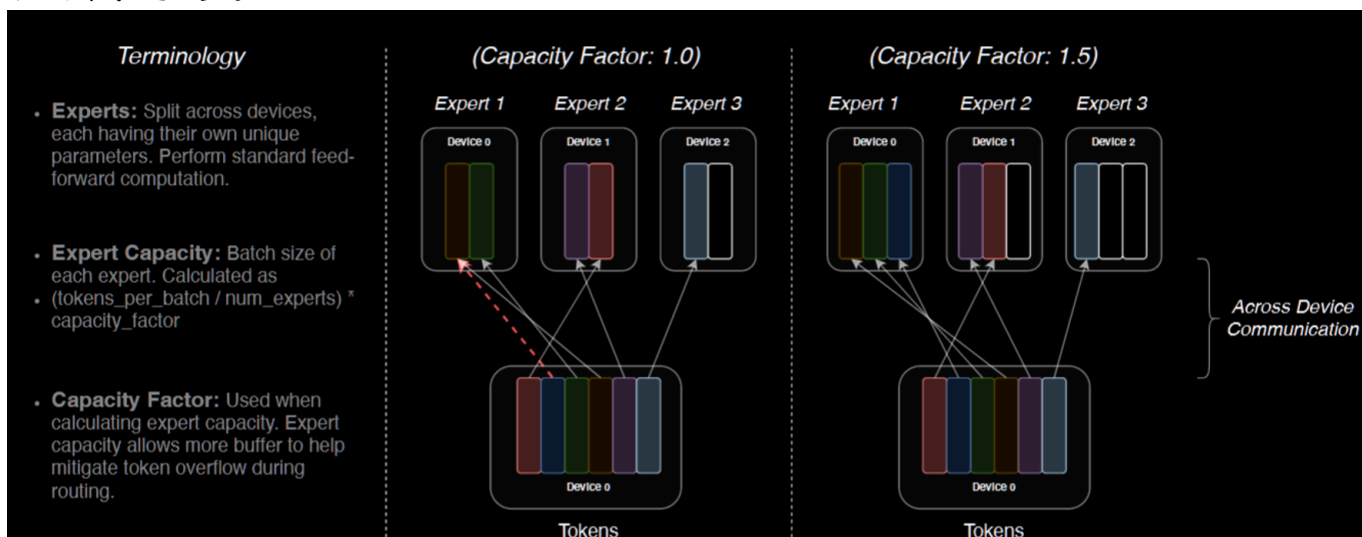
专家容量：

$$\text{experts Capacity} = \text{Factor} * \frac{\text{samples}}{\text{experts}}$$

- 如果Factor = 1，那么就是严格的平均分
- Factor > 1，就会存在容错，专家分配可能小部分不均衡

如果专家溢出，就会直接跳过专家这一步，进行残差连接

设置合适的专家容量很重要，过多会导致多卡不均衡和内存占用过多。过少会导致过多样本直接跳过专家这一步。



软约束

同样的想法，也就是辅助损失函数

$$loss = \alpha \cdot N \cdot \sum_{i=1}^N f_i P_i$$

- α : 损失权重超参数（乘法系数），用于平衡“负载均衡目标”与“主任务目标，确定最优值为 $\alpha = 10^{-2}$
- N : 专家数量（如 Switch-Base 常用 128 个专家）。乘以N的目的是让损失值在专家数量变化时保持稳定
- f_i : 第i个专家的“token 分配比例”（实际接收的 token 占比）;
- P_i : 第i个专家的“概率分配比例”（路由器为其分配的概率质量占比）;
- $\sum_{i=1}^N f_i \cdot P_i$: f向量与P向量的点积，用于衡量“实际 token 分配”与“路由器概率分配”的偏差 —— 点积越小，说明两者越接近均匀分布，负载越均衡。
该辅助函数用向量相乘的思想，最小化被跳过的samples。

Dropout

注意到，对于每一个专家，实际上训练的样本比正常的FFN小，因为样本被分散了。因此有了一个朴素的思想，就是对于专家的正则化要狠一点。

在微调的时候也一样。

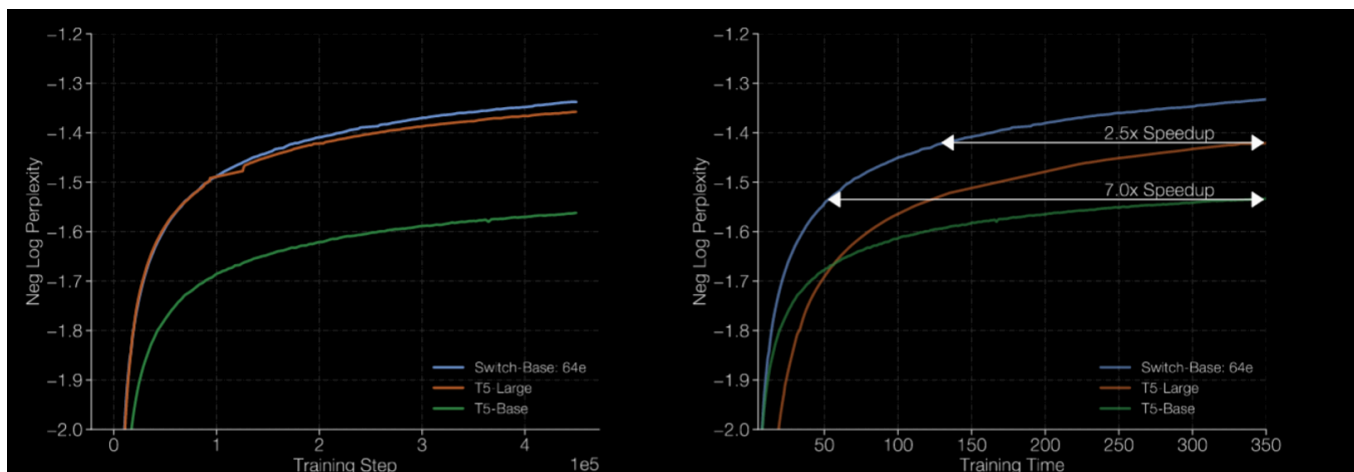
论文中提出了**专家Dropout**，文中给出的最佳是 **ed (expert dropout) = 0.4** , **d = 0.1**

同时要注意，并不是Dropout越高，对于模型泛化能力越好，因为他起到一个破坏的作用，如果对于一个表单能力合适的模型，过高的Dropout会破坏提取特征的能力。

Model (dropout)	GLUE	CNNDM	SQuAD	SuperGLUE
T5-Base (d=0.1)	82.9	19.6	83.5	72.4
Switch-Base (d=0.1)	84.7	19.1	83.7	73.0
Switch-Base (d=0.2)	84.4	19.2	83.9	73.2
Switch-Base (d=0.3)	83.9	19.6	83.4	70.7
Switch-Base (d=0.1, ed=0.4)	85.2	19.6	83.7	73.0

Table 4: Fine-tuning regularization results. A sweep of dropout rates while fine-tuning Switch Transformer models pre-trained on 34B tokens of the C4 data set (higher numbers are better). We observe that using a lower standard dropout rate at all non-expert layer, with a much larger dropout rate on the expert feed-forward layers, to perform the best.

稀疏or密集



1. 模型参数开销:

两个base的运算速度一致, 但是, 参数更多的switch显然更好

2. 模型计算量/速度开销:

Large和switch看出在相同参数开销的时候困惑度都表现很好, 但是在计算速度上switch还是更胜一筹。

总结, 并联的参数增加相较于深度堆叠更有优势。深度神经网络->广度+深度神经网络。